



Lista 2 — Cloud Native, DevOps e Microserviços

Aulas 09 a 16

Tópicos Avançados em TI

Exercícios sobre Kubernetes, microserviços, API Gateway, DevOps, CI/CD, GitOps e arquitetura moderna.
Cenários de deploy em cloud e operação de infraestrutura.

Exercício 1

Cenário: Migração de monólito para microserviços — empresa de e-commerce com 500k pedidos/mês.

1. Identifique os bounded contexts (DDD): catálogo, carrinho, pedidos, pagamento, notificação, usuários. Desenhe a arquitetura de microserviços proposta.
2. Para cada serviço, defina: tecnologia (Node.js, Go, Python), banco de dados (Postgres, Redis, MongoDB), comunicação (REST, gRPC, Kafka).
3. Implemente o padrão Saga coreografado para o fluxo de compra: reserva de estoque → pagamento → envio → notificação. Descreva o evento de compensação (rollback) para cada etapa.
4. Configure um API Gateway (Kong ou NGINX) com: roteamento para cada serviço, rate limiting (100 req/min por cliente), autenticação JWT.

Exercício 2

Cenário: Pipeline CI/CD com deploy em Kubernetes (minikube ou kind).

1. Crie um Dockerfile multi-stage para uma app Node.js: stage 1 (build com devDeps), stage 2 (apenas runtime, imagem < 200MB).
2. Escreva manifestos Kubernetes: Deployment (3 réplicas, resource limits 256m CPU / 512Mi RAM), Service (ClusterIP :80), ConfigMap, Secret.
3. Configure GitHub Actions: workflow com lint → test → build → push Docker Hub → kubectl apply no cluster.
4. Implemente GitOps com ArgoCD: repositório Git como fonte da verdade, ArgoCD sincroniza automaticamente o estado do cluster com o repositório.

Exercício 3

Cenário: Observabilidade — monitorar sistema com dezenas de microsserviços.

1. Implemente os 3 pilares da observabilidade: logs (estruturados em JSON, coletados pelo Loki/ELK), métricas (Prometheus exporta CPU/RAM/requests, dashboards Grafana), tracing distribuído (OpenTelemetry + Jaeger).
2. Defina SLIs (disponibilidade, latência p99, taxa de erro) e SLOs correspondentes (ex: 99.9% disponibilidade, p99 < 200ms, erro < 0.1%).
3. Configure alertas no Prometheus: regras de alerta para SLOs violados, Alertmanager envia notificações para Slack/PagerDuty.
4. Conceito de error budget: se o SLO é 99.9%, o error budget mensal é 0.1% = 43 min. Como decidir se novas funcionalidades podem ser lançadas baseado no error budget restante?

Exercício 4

Cenário: Infraestrutura como Código (IaC) com Terraform para cloud (AWS, GCP ou Azure).

1. Escreva Terraform para provisionar: VPC, subnets públicas e privadas, security groups, instância EC2 (ou VM equivalente) com Nginx, RDS (Postgres).
2. Use módulos do Terraform Registry (ex: terraform-aws-modules/vpc/aws) para organizar o código.
3. Gerencie estado remoto do Terraform no S3 (AWS) com DynamoDB para locking.
4. Compare Terraform com Pulumi e AWS CDK: qual a diferença entre HCL e linguagens de programação (TypeScript, Python) para IaC?

Requisitos

Docker, minikube (ou kind), kubectl. Terraform (opcional, conta AWS gratuita). GitHub para Actions. ArgoCD (opcional).